
OSR TRNG FPGA IMPLEMENTATION GUIDE

IMPLEMENTATION GUIDE

文档信息

版本	v1.0
交付客户	
关键字	TRNG
摘要	该文档主要描述 OSR TRNG 的 FPGA 实现指导

Copyright Notice and Proprietary Information Notice

Copyright © 2014 - 2023 Open Security Research, Inc. All rights reserved. This documentation contains confidential and proprietary information that is the property of Open Security Research, Inc. The documentation is transported under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, translated, distributed, disclosed, announced or copied in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Open Security Research, Inc., or as expressly provided by the license agreement.

版权声明和专有信息声明

版权所有©2014-2023 深圳市纽创信安科技发展有限公司 保留所有权利。本文档包含机密和专有信息，属于深圳市纽创信安科技发展有限公司的财产。该文档根据对应的许可协议传播，且只能根据许可协议的条款使用或复制。未经深圳市纽创信安科技发展有限公司事先书面许可或经许可协议明确规定，该文档和对应软件的全部或任何部分均不得以任何形式或方式（电子的，机械的，手动的，光学的或其他的）被复制、传播、翻译、分发、披露、宣布或抄袭。

历史版本

版本号	日期	描述
1.0	2023/08/03	正式版第一版

目录

IMPLEMENTATION GUIDE	1
1 文档概述	5
1.1 总体概述	5
1.2 章节介绍	5
2 熵源结构说明	6
2.1 RO 快环熵源结构	6
2.2 RO_CLK 慢环结构	6
3 FPGA 结构约束	8
4 FPGA 时序约束	9
5 环路时序分析	10
5.1 快环时序报告	10
5.2 慢环时序报告	11
5.3 时序不如预期处理方法	12
5.3.1 替换反相器器件	12
5.3.2 添加 PBLOCK	13
5.3.3 熵源环路频率调整	15

1 文档概述

1.1 总体概述

由于 TRNG 熵源部分为大量组合逻辑环路组成的随机震荡源的特性，其在综合、FPGA 测试与后端实现都有根据环境进一步调试的需求。在 FPGA 实现时，组合逻辑环结构可能被工具优化，导致无随机数输出、输出全 0 随机数或只有部分环路可正常输出等问题，本文档主要介绍在 FPGA 实现时需要需要的操作和取数不成功时 debug 的流程。如仍遇到其他问题，可联系 OSR 进行技术支持。

1.2 章节介绍

本文档正文部分共分为 4 个章节，其中第 2 章节简述了熵源部分快环和慢环的结构，与其产生随机数的原理。

第 3 章与第 4 章介绍了在 FPGA 上实现时所需要的保持结构的约束和时序约束。

第 5 章介绍如 FPGA 实现不按预期时通过 vivado 工具 debug 的可用方法。

2 熵源结构说明

本章节介绍熵源部分设计。TRNG 模块有四个相互独立的 RO 熵源，每个 RO 熵源包含 16 路 RO 快环和 1 路慢环，即采样环。其结构是慢环采样时钟经过分频后去采样快环熵源时钟，并按分频后的频率输出随机数。

2.1 RO 快环熵源结构

HDL 文件名:

- ro_circuit.v

模块名:

- ro_circuit

例化名前缀:

- OSR_DONTTOUCH_

电路图:

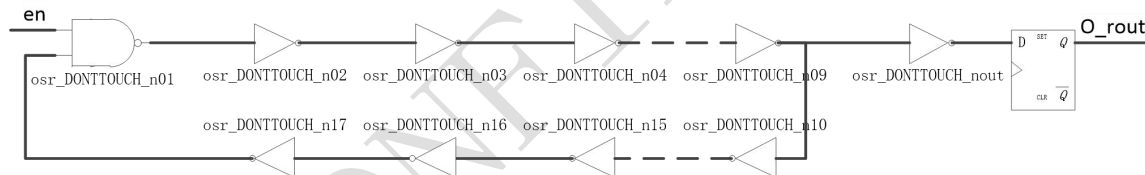


图 2-1 OSR RO CIRCUIT SCHEMATIC

熵源部分共包括 64 路上述 RO 环，称为快环，提供随机 bit 源。

2.2 RO_CLK 慢环结构

HDL 文件名:

- ro_clk_circuit.v

模块名:

- ro_clk_circuit

例化名前缀:

- OSR_DONTTOUCH_

电路图:

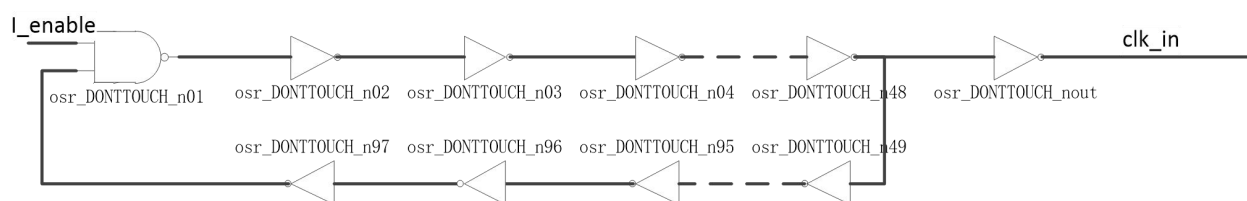


图 2-2 OSR RO CLK CIRCUIT SCHEMATIC

熵源部分共包括 4 路上述 RO_CLK 环，称为慢环，以其分频后的频率采样快环并按分频后的频率从震荡源中取随机数。

3 FPGA 结构约束

TRNG 中熵源部分的 Combinational Loop 在 VIVADO Write Bitstream 时会报错，建议在 Write Bitstream 时添加以下指令：

```
# ignore combinational loops drc error

set_property ALLOW_COMBINATORIAL_LOOPS TRUE [get_nets -hier -regexp
"./u_ro/ros\[1-4\]/.*OSR_DONTTOUCH_n/Z"]
```

熵源采用反相器构建组合逻辑环路，因此存在可能被 FPGA 工具优化掉的风险，需要手动添加 donttouch 属性。方式如下：

- 在 rtl 代码中已在快环熵源模块 ro_circuit.v 和慢环熵源模块 ro_clk_circuit.v 中加入防止信号被优化的标识，分别为“(*dont_touch = "true"*)”和“/* synthesis syn_keep=1 */”。

两个模块内的标识均如下，请按照实际应用的 FPGA 平台使用相应的命令对信号进行约束，防止信号被编译器优化。

```
(*dont_touch = "true"*) wire [INV_N-1:0] inv_n /* synthesis syn_keep=1 */;
(*dont_touch = "true"*) wire inv_0 /* synthesis syn_keep=1 */;
(*dont_touch = "true"*) wire inv_1 /* synthesis syn_keep=1 */;
(*dont_touch = "true"*) wire clk_in /* synthesis syn_keep=1 */;
(*dont_touch = "true"*) wire inv_t /* synthesis syn_keep=1 */;
(*dont_touch = "true"*) wire inv_in /* synthesis syn_keep=1 */;
(*dont_touch = "true"*) reg dft reg /* synthesis syn_keep=1 */;
```

图 3-1 FPGA 信号保持约束示例

4 FPGA 时序约束

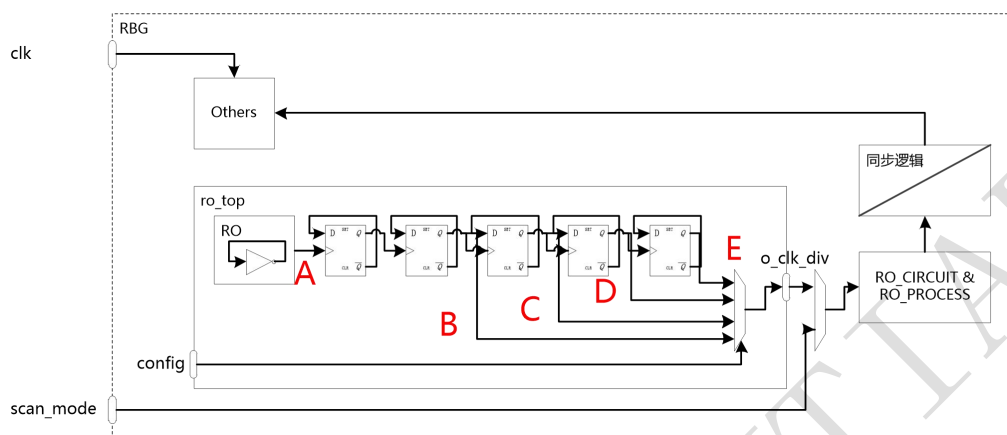


图 4-1 RO 熵源结构示意图

对于 FPGA 的序约束，建议如下：

- 需要对组合逻辑环路设置 `set_disable_timing`，可分别在快环与慢环起点即 `OSR_DONTTOUCH_n01/Z` 与 `OSR_DONTTOUCH_n1/Z` 处分别设置。参考语句：

```
set_disable_timing [get_pins -hier -regexp ".*ros\[1-4\]/.*OSR_DONTTOUCH_n01/Z"]
set_disable_timing [get_pins -hier -regexp ".*ros\[1-4\]/.*OSR_DONTTOUCH_n1/Z"]
```

- 在 `ro_clk_circuit` 模块的 RO 环输出（即标号 A 处）定义时钟，即 `create_clock`。频率可以根据 RO 环内反相器个数结合工艺库内延迟参数来确定。
- 在 `ro_clk` 模块的异步分频器的标号 B, C, D 及 E 分别生成 2 分频时钟，即 `create_generated_clock`。
- 在 `ro_clk` 的输出端口 `o_clk_div` 设置生成时钟。
- 设置时钟组，将 RO 时钟域内的时钟，即 A, B, C, D 及 E 处的时钟归为一组，将系统时钟 `clk` 设为另一组，二组关系为异步。可参考 `set_clock_groups -asynchronous -group`。

至此，熵源已加上了时序约束其内部逻辑会根据所设定的 `ro_clk` 时钟进行 STA 分析，其他模块则根据系统时钟 `clk` 进行分析。两时钟域间的路径为伪路径，不需要分析。

OSR_TRNG 内含有 4 个独立熵源，每个熵源均需要按照上述方式进行约束。

5 环路时序分析

如 FPGA 测试结果不按预期（API 不能正常运行），可在 vivado 工具中分别报出快环和慢环时序信息辅助分析。期望结果为慢环的总延时与快环总延时的预期比例约为 6: 1（即约为 5.3.3 章节设置的慢环和快环中反相器个数的比）。

5.1 快环时序报告

如 FPGA 测试结果不按预期，可在 vivado 工具中分别报出快环和慢环时序信息辅助分析。结构未被优化情况下，快环网表结构应如下：

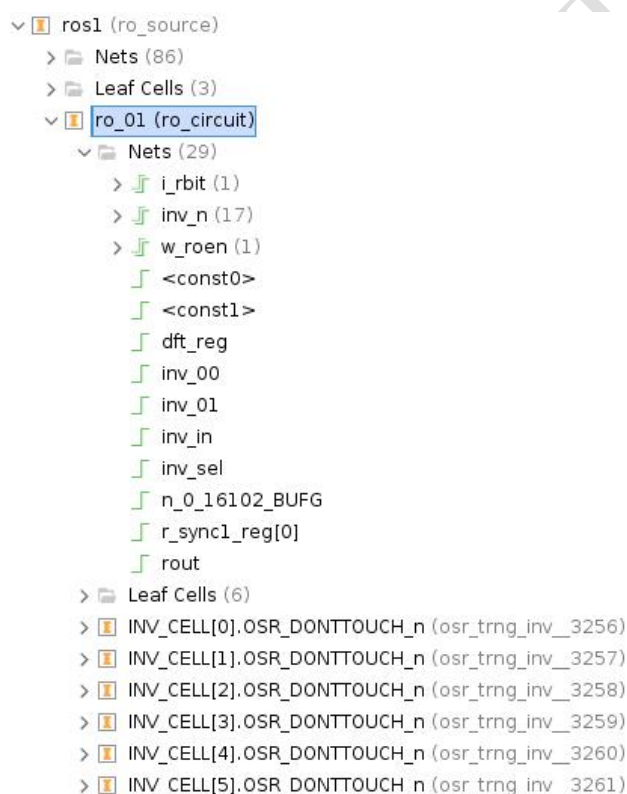


图 5-1 快环网表结构示意图

其中 inv_n(17)数量为 5.3.3 章节参数调整的快环反相器数量。

以图中 ro_01 快环为例，进入 ro_01(ro_circuit)的 schematic，可看到快环反相器链路。在报告时序信息时推荐采用先设置链路的最大或最小延时以提示链路的存在。可借助代码中标注 donttouch 保持的 inv_n,inv_00 和 inv_01 信号设置最大或最小延时。参考语句：

```
set_min_delay 3 -from \
[get_pins u_osr_trng_top/u_trbg_top/u_ro/ros1/ro_01/inv_01_inst/I0] \
-to [get_pins u_osr_trng_top/u_trbg_top/u_ro/ros1/ro_01/inv_n_inst/O]
```

All rights reserved.

参考语句中的 `u_osr_trng_top` 为例化 `OSR_TRNG` 时例化名，可根据实际调整。接着在链路中的一点 `report_timing` 即可得到环路的总延时。示例如下：

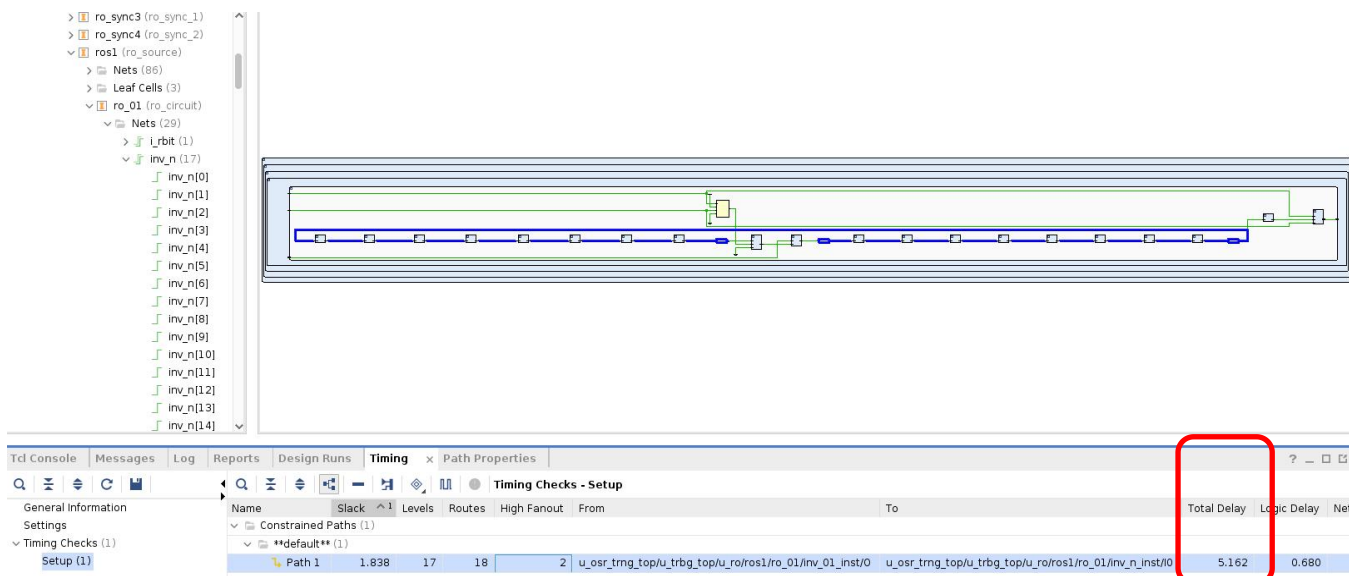


图 5-2 快环 `report_timing` 示例

5.2 慢环时序报告

结构未被优化情况下，快环网表结构应如下：

```

roclk (ro_clk)
├── Nets (18)
├── Leaf Cells (12)
└── ro (ro_clk_circuit)
    ├── Nets (108)
    │   ├── inv_n (97)
    │   ├── Q (1)
    │   ├── <const0>
    │   ├── <const1>
    │   ├── CLK
    │   ├── clk_in
    │   ├── dft_reg
    │   ├── dft_reg_reg_0
    │   ├── inv_0
    │   ├── inv_1
    │   ├── inv_in
    │   └── inv_t
    ├── Leaf Cells (6)
    │   ├── INV_CELL[0].OSR_DONTTOUCH_n (osr_trng_inv_2860)
    │   ├── INV_CELL[1].OSR_DONTTOUCH_n (osr_trng_inv_2861)
    │   ├── INV_CELL[2].OSR_DONTTOUCH_n (osr_trng_inv_2862)
    │   ├── INV_CELL[3].OSR_DONTTOUCH_n (osr_trng_inv_2863)
    │   ├── INV_CELL[4].OSR_DONTTOUCH_n (osr_trng_inv_2864)
    │   ├── INV_CELL[5].OSR_DONTTOUCH_n (osr_trng_inv_2865)
    │   └── INV_CELL[6].OSR_DONTTOUCH_n (osr_trng_inv_2866)

```

图 5-3 慢环网表结构示意图

其中 `inv_n(97)` 数量为 5.3.3 章节参数调整的快环反相器数量。

以图中 `roclk` 慢环为例，其中包含的 **Nets** 和 **Leaf Cells** 为慢环的分频逻辑。进入 `ro(ro_clk_circuit)` 的 **schematic**，可看到快环反相器链路。在报告时序信息时推荐采用先设置链路的最大或最小延时以提示链路的存在。可借助代码中标注 `donttouch` 保持的 `inv_n, inv_00` 和 `inv_01` 信号设置最大或最小延时。参考语句：

```
set_min_delay 3 -from \
[get_pins u_osr_trng_top/u_trbg_top/u_ro/ros1/roclk/ro/inv_0_inst/I0] \
-to [get_pins u_osr_trng_top/u_trbg_top/u_ro/ros1/roclk/ro/inv_n_inst/O]
```

参考语句中的 `u_osr_trng_top` 为例化 `OSR_TRNG` 时例化名，可根据实际调整。

接着在链路中的一点 `report_timing` 即可得到环路的延时。示例如下：

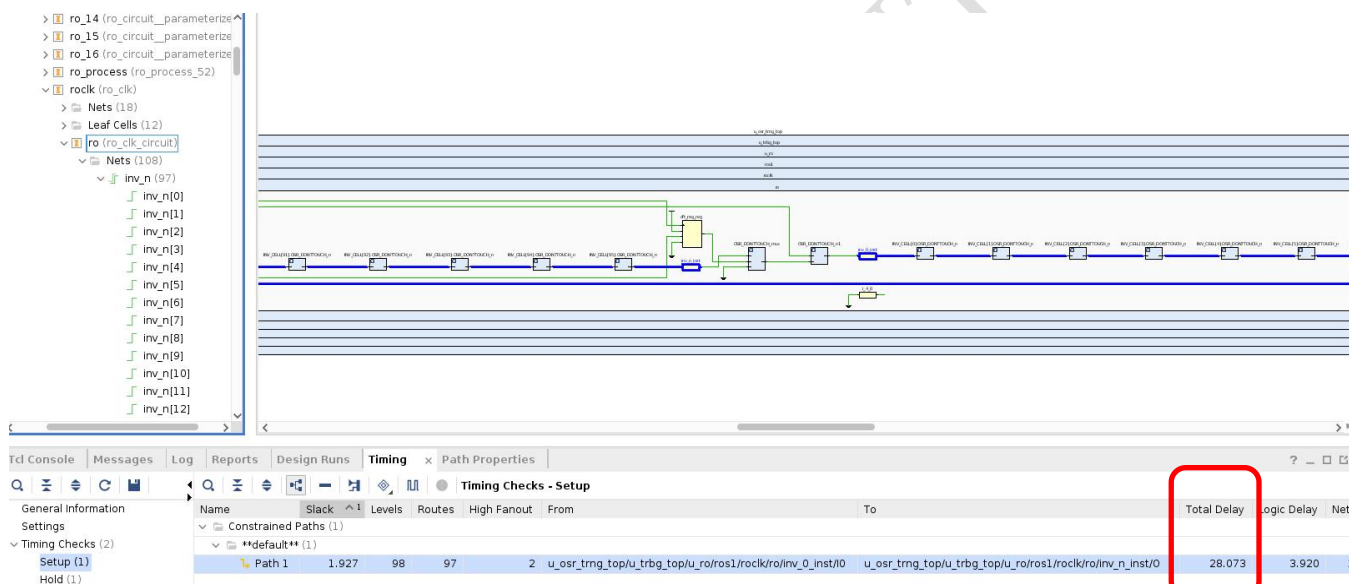


图 5-4 慢环 `report_timing` 示例

5.3 时序不如预期处理方法

5.3.1 替换反相器器件

如快慢环层次结构不包含参数相应数量的反相器或慢环与快环延时远远偏离 **6:1** 则说明组合逻辑环路被优化。应先检查前文中 **FPGA** 结构约束与时序约束是否已添加。如确认均已添加则进行如下操作：

- 查找 `./rtl/tokens/trng/general/logic_cell.v` 文件
- 打开后可以看到 **FPGA** 宏定义
- 将 **FPGA** 宏定义下的逻辑替换为 **vivado** 工具库中相应 **LUT** 资源，并通过参数设置 **LUT** 功能

All rights reserved.

- 保存文件即可退出

5.3.2 添加 PBLOCK

TRNG 是产生随机数的模块,在 FPGA 综合过程中有可能会将其 RO 与其他模块的 RAM、LUT 等资源布局在一块区域;会对随机数的产生,造成不确定影响,因此需要在 FPGA 综合后使用 PBLOCK,划定区域专门用于 RO 环的摆放,下面以 vivado 为例展开,实际添加的 PBLOCK 时可根据本身设计及工艺灵活设置:

1、进行工程综合

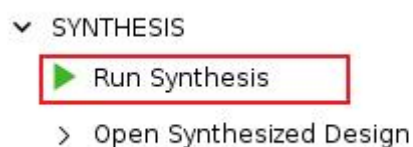


图 5-5-1 PBLOCK 流程 1

2、等待综合完成

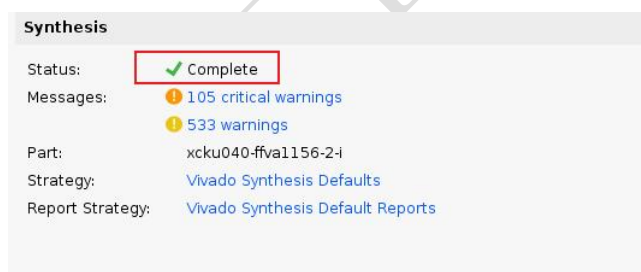


图 5-5-2 PBLOCK 流程 2

3、打开综合后设计



图 5-5-3 PBLOCK 流程 3

4、在 netlist 中找到 trng_ro_top, 这就是 ro 环所在的模块。右键选择 floorplanning, 点击 draw pblock。

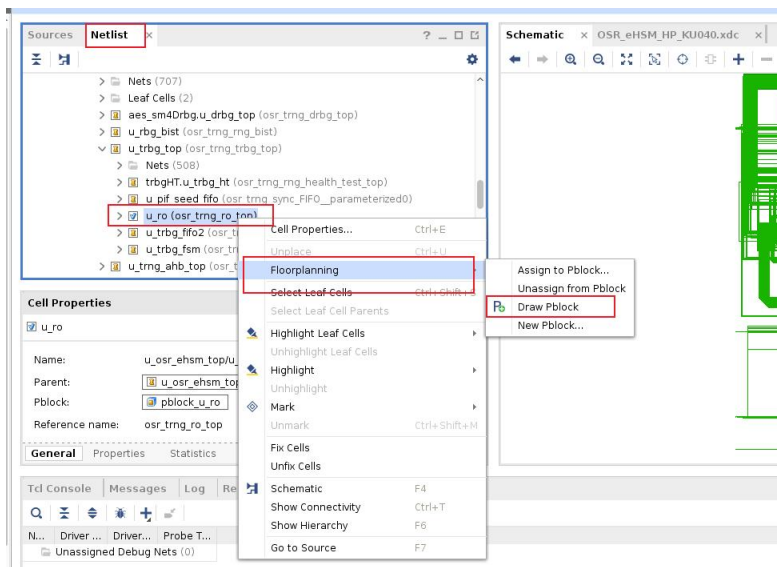


图 5-5-4 PBLOCK 流程 4

- 5、划定选定的区域用于 ro 环的布置，将所有资源打勾，那么这块区域所有资源将只会用于 ro 环的 floorplan

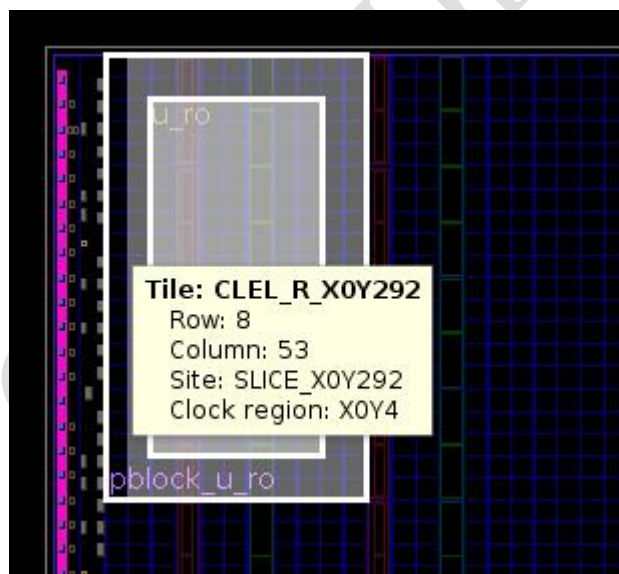


图 5-5-5 PBLOCK 流程 5



图 5-5-6 PBLOCK 流程 6

6、ctrl+s 保存后，这块区域会变为约束保存在 xdc 中。

```
create_pblock pblock_u_ro_1
add_cells_to_pblock [get_pblocks pblock_u_ro_1] [get_cells -quiet [list u_osr_ehsm_top/u_ahb_wrapper/u_trng/u_trng_top/u_ro]]
resize_pblock [get_pblocks pblock_u_ro_1] -add {SLICE_X11Y275:SLICE_X23Y293}
resize_pblock [get_pblocks pblock_u_ro_1] -add {DSP48E2_X1Y110:DSP48E2_X3Y115}
resize_pblock [get_pblocks pblock_u_ro_1] -add {RAMB18_X2Y110:RAMB18_X2Y115}
resize_pblock [get_pblocks pblock_u_ro_1] -add {RAMB36_X2Y55:RAMB36_X2Y57}
```

7、由此完成所有步骤，这样约束基本能解决随机数生成质量检测不过的问题；同时会浪费一些 FPGA 的资源面积作为代价。

8、生成 bit，需要忽略 trng 中的 looping，设置以下约束。

```
set_property SEVERITY {Warning} [get_drc_checks LUTLP-1]
```

5.3.3 熵源环路频率调整

如 FPGA 取出随机数为全 0，可能的原因之一为先进工艺的 FPGA 反相器引入的延时太小，按照代码提供的默认参数下环路引入延时小，导致震荡环的预期频率很高，超过 FPGA 器件可起振上限。

因此如综合后未发现结构被优化的现象，且已添加 PBLOCK 后仍无法正常取出除随机数，可考虑调整代码中的参数改变环内反相器个数，以使震荡环频率降低。下图列出了如何调整快环、慢环的级数。如果输出随机数为全零则说明慢环起振，调整方法为可在快环与慢环反相器个数参数处乘以 3 或 5，如慢环也未起振则需乘以 7 甚至更多。调整时应保证：

- 快环与慢环反相器个数比例约为 6:1

All rights reserved.

- 所有参数均设置为奇数
- 16 路快环之间或 4 路慢环之间反相器个数有差异

● 调整 RO 快环电路级数

RTL: rtl/tokens/trng/ro_core/ro_source.v

```
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+0), .INV_N(17), .delay(810))ro_01(.en(i_roen[0]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[0]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+1), .INV_N(23), .delay(830))ro_02(.en(i_roen[1]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[1]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+2), .INV_N(19), .delay(570))ro_03(.en(i_roen[2]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[2]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+3), .INV_N(25), .delay(870))ro_04(.en(i_roen[3]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[3]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+4), .INV_N(21), .delay(890))ro_05(.en(i_roen[4]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[4]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+5), .INV_N(17), .delay(710))ro_06(.en(i_roen[5]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[5]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+6), .INV_N(23), .delay(730))ro_07(.en(i_roen[6]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[6]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+7), .INV_N(19), .delay(590))ro_08(.en(i_roen[7]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[7]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+8), .INV_N(25), .delay(770))ro_09(.en(i_roen[8]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[8]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+9), .INV_N(21), .delay(790))ro_10(.en(i_roen[9]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[9]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+10), .INV_N(17), .delay(610))ro_11(.en(i_roen[10]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[10]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+11), .INV_N(23), .delay(630))ro_12(.en(i_roen[11]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[11]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+12), .INV_N(19), .delay(670))ro_13(.en(i_roen[12]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[12]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+13), .INV_N(25), .delay(690))ro_14(.en(i_roen[13]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[13]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+14), .INV_N(21), .delay(510))ro_15(.en(i_roen[14]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[14]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+15), .INV_N(17), .delay(530))ro_16(.en(i_roen[15]), .clk(w_clk), .rstn(w_rst_n), .scan(i_scan), .o_rout(w_rbit0[15]));
```

图 5-9 RO 快环级数调整

● 调整 RO 慢环电路级数

RTL: rtl/tokens/trng/ro_core/ro_top.v

```
osr_trng_ro_source #(.p_IDX(0), .CLK_INV_N(97), .delay(5010))
ros1(
    .clk          (clk          ),
    .resetn       (resetn       ),
    .ro_rst       (w_ro_rst1    ),
    .i_scan       (i_scan       ),
    .i_roen       (w_roen[3]    ),
    .i_conf       (w_conf       ),
    .i_roen       (w_roen[63:48]),
    .o_rand       (w_rand1      ),
    .i_flag       (w_iflag1     ),
    .o_flag       (w_oflag1     ),
    .o_rornd      (w_rornd1     ),
    .o_roclk      (w_roclk1     )
);

osr_trng_ro_source #(.p_IDX(1), .CLK_INV_N(95), .delay(4770))
ros2(
    .clk          (clk          ),
    .resetn       (resetn       ),
    .ro_rst       (w_ro_rst2    ),
    .i_scan       (i_scan       ),
    .i_roen       (w_roen[2]    ),
    .i_conf       (w_conf       ),
    .i_roen       (w_roen[47:32]),
    .o_rand       (w_rand2      ),
    .i_flag       (w_iflag2     ),
    .o_flag       (w_oflag2     ),
    .o_rornd      (w_rornd2     ),
    .o_roclk      (w_roclk2     )
);
```



```
osr_trng_ro_source #(.p_IDX(2), .CLK_INV_N(93) .delay(4510))
ros3(
  .clk      (clk      ),
  .resetn   (resetn   ),
  .ro_rst   (w_ro_rst3 ),
  .i_scan   (i_scan   ),
  .i_rose   (w_rose[1] ),
  .i_conf   (w_conf   ),
  .i_roen   (w_roen[31:16] ),
  .o_rand   (w_rand3   ),
  .i_flag   (w_iflag3  ),
  .o_flag   (w_oflag3  ),
  .o_rornd  (w_rornd3  ),
  .o_roclk  (w_roclk3  )
);

osr_trng_ro_source #(.p_IDX(3), .CLK_INV_N(91) .delay(4170))
ros4(
  .clk      (clk      ),
  .resetn   (resetn   ),
  .ro_rst   (w_ro_rst4 ),
  .i_scan   (i_scan   ),
  .i_rose   (w_rose[0] ),
  .i_conf   (w_conf   ),
  .i_roen   (w_roen[15: 0] ),
  .o_rand   (w_rand4   ),
  .i_flag   (w_iflag4  ),
  .o_flag   (w_oflag4  ),
  .o_rornd  (w_rornd4  ),
  .o_roclk  (w_roclk4  )
);
```

图 5-10 RO 慢环级数调整